

main.c

InnoIDE Reactive Node Firmware · ESP32-S3 · 789 lines · Generated 2026-09-17

```
1  #include <stdio.h>
2  #include <stdint.h>
3  #include <stdbool.h>
4  #include <stddef.h>
5  #include <string.h>
6  #include <ctype.h>
7  // sys/types.h and sys/param.h are intentionally omitted:
8  // lwIP / ESP-IDF provide their own ssize_t typedef (as long long).
9  // Including sys/types.h before lwIP headers causes a typedef redefinition error.
10
11 #include "freertos/FreeRTOS.h"
12 #include "freertos/task.h"
13 #include "freertos/event_groups.h"
14
15 #include "esp_system.h"
16 #include "esp_event.h"
17 #include "esp_log.h"
18 #include "esp_netif.h"
19 #include "esp_wifi.h"
20 #include "nvs_flash.h"
21
22 #include "driver/gpio.h"
23 #include "driver/usb_serial_jtag.h"
24 #include "driver/uart.h"
25 #include "led_strip.h"
26
27 #include "lwip/err.h"
28 #include "lwip/sockets.h"
29 #include "lwip/sys.h"
30 #include "lwip/netdb.h"
31 #include "esp_http_server.h"
32
33 static const char *TAG = "INNO_REACTIVE_NODE";
34
35 // =====
36 // Wi-Fi Configuration
37 // You can edit these constants to match your local 2.4 GHz Wi-Fi router.
38 // If the router cannot be reached after 5 attempts, the ESP32 automatically
39 // spins up its own SoftAP hotspot ("InnoIDE-ESP32-S3") for direct connection.
40 // =====
41 #define DEFAULT_WIFI_SSID "innotrat"
42 #define DEFAULT_WIFI_PASS "Innovate@91761"
43 #define FALLBACK_AP_SSID "InnoIDE-ESP32-S3"
44 #define FALLBACK_AP_PASS "12345678"
45 #define TCP_SERVER_PORT 8888
46 #define UDP_BEACON_PORT 5056
47
48 // Hardware LED Pins for ESP32-S3 DevKit
49 #define LED_GPIO_WS2812 GPIO_NUM_48
50 #define LED_GPIO_STANDARD GPIO_NUM_2
51 #define LED_COUNT 1
52
53 static led_strip_handle_t s_led_strip = NULL;
54 static volatile int s_led_state = 0; // 0 = OFF (STOPPED), 1 = ON (BLINKING)
```

```
55
56 // Network state variables
57 static volatile int s_wifi_connected = 0;
58 static char s_device_ip[32] = "0.0.0.0";
59 static char s_wifi_mode[16] = "DISCONNECTED";
60 static volatile int s_active_client_sock = -1;
61 static httpd_handle_t s_http_server = NULL;
62
63 static void send_serial_response(const char *msg)
64 {
65 // 1. Send over USB-Serial/JTAG (Native USB CDC)
66 if (usb_serial_jtag_is_connected()) {
67     usb_serial_jtag_write_bytes(msg, strlen(msg), pdMS_TO_TICKS(100));
68 }
69 // 2. Send over standard UART0 (TX pin)
70 uart_write_bytes(UART_NUM_0, msg, strlen(msg));
71 printf("%s", msg);
72 fflush(stdout);
73
74 // 3. Send over active Wi-Fi TCP socket if connected
75 int sock = s_active_client_sock;
76 if (sock >= 0) {
77     send(sock, msg, strlen(msg), 0);
78 }
79 }
80
81 static void apply_led_hardware(int on)
82 {
83 if (s_led_strip) {
84     if (on) {
85 // Vibrant Cyan-Blue (RGB = 0, 180, 255)
86 led_strip_set_pixel(s_led_strip, 0, 0, 180, 255);
87 led_strip_refresh(s_led_strip);
88 } else {
89 // Completely OFF
90 led_strip_clear(s_led_strip);
91 led_strip_refresh(s_led_strip);
92 }
93 }
94 gpio_set_level(LED_GPIO_STANDARD, on ? 1 : 0);
95 }
96
97 // Background task to blink the LED rhythmically when s_led_state == 1
98 static void led_controller_task(void *pvParameters)
99 {
100     int blink_phase = 0;
101
102     while (1) {
103         if (s_led_state == 1) {
104             // Active / Blinking state (350ms ON, 350ms OFF)
105             blink_phase = !blink_phase;
106             apply_led_hardware(blink_phase);
107             vTaskDelay(pdMS_TO_TICKS(350));
108         } else {
109             // Stopped / Completely OFF
110             blink_phase = 0;
111             apply_led_hardware(0);
112             vTaskDelay(pdMS_TO_TICKS(50));
113         }
114     }
115 }
```



```
173 send_serial_response(resp);
174 }
175 // Ping / Heartbeat
176 else if (strcmp(clean, "PING") == 0)
177 {
178     send_serial_response("PONG\r\n");
179 }
180 // Reboot
181 else if (strcmp(clean, "REBOOT") == 0 || strcmp(clean, "RESET") == 0)
182 {
183     send_serial_response("REBOOTING...\r\n");
184     vTaskDelay(pdMS_TO_TICKS(200));
185     esp_restart();
186 }
187 // Wi-Fi Air Scanner
188 else if (strcmp(clean, "SCAN") == 0 || strcmp(clean, "WIFI:SCAN") == 0)
189 {
190     wifi_scan_config_t scan_config = { 0 };
191     esp_wifi_scan_start(&scan_config, true);
192     uint16_t ap_count = 0;
193     esp_wifi_scan_get_ap_num(&ap_count);
194     if (ap_count > 0) {
195         wifi_ap_record_t *ap_list = malloc(sizeof(wifi_ap_record_t) * ap_count);
196         if (ap_list) {
197             esp_wifi_scan_get_ap_records(&ap_count, ap_list);
198             send_serial_response("--- WIFI NETWORKS DETECTED ---\r\n");
199             for (int i = 0; i < ap_count && i < 15; i++) {
200                 char scan_buf[96];
201                 snprintf(scan_buf, sizeof(scan_buf), "SSID: %-22s RSSI:%d dBm\r\n",
202                     (char *)ap_list[i].ssid, ap_list[i].rssi);
203                 send_serial_response(scan_buf);
204             }
205             send_serial_response("--- END SCAN ---\r\n");
206             free(ap_list);
207         }
208     } else {
209         send_serial_response("SCAN: No Wi-Fi networks found\r\n");
210     }
211 }
212 else
213 {
214     char resp[128];
215     snprintf(resp, sizeof(resp), "ACK:%s (LED=%d, IP=%s)\r\n", clean, s_led_state, s_device_ip);
216     send_serial_response(resp);
217 }
218 }
219
220 // =====
221 // Embedded HTTP REST Server Handlers
222 // =====
223 static esp_err_t http_action_handler(httpd_req_t *req)
224 {
225     char buf[128] = {0};
226     int ret = 0;
227
228     // Check if GET query string provided (?payload=LED:1)
229     if (req->method == HTTP_GET) {
230         if (httpd_req_get_url_query_str(req, buf, sizeof(buf)) == ESP_OK) {
231             char param[64] = {0};
```

```
232 if (httpd_query_key_value(buf, "payload", param, sizeof(param)) == ESP_OK) {
233     process_incoming_command(param);
234 } else if (httpd_query_key_value(buf, "cmd", param, sizeof(param)) == ESP_OK) {
235     process_incoming_command(param);
236 }
237 }
238 } else if (req->method == HTTP_POST) {
239     int total_len = req->content_len;
240     if (total_len > 0 && total_len < sizeof(buf)) {
241         ret = httpd_req_recv(req, buf, total_len);
242         if (ret > 0) {
243             buf[ret] = '\0';
244             // Look for "payload":"LED:1" in JSON or raw string
245             char *p = strstr(buf, "LED:1");
246             if (p) {
247                 process_incoming_command("LED:1");
248             } else if ((p = strstr(buf, "LED:0")) != NULL) {
249                 process_incoming_command("LED:0");
250             } else {
251                 process_incoming_command(buf);
252             }
253         }
254     }
255 }
256
257 httpd_resp_set_type(req, "application/json");
258 httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "");
259 char resp[128];
260 snprintf(resp, sizeof(resp), "{\"success\":true,\"led\":%d,\"state\":\"%s\"}",
261     s_led_state, s_led_state ? "ON" : "OFF");
262 httpd_resp_send(req, resp, HTTPD_RESP_USE_STRLEN);
263 return ESP_OK;
264 }
265
266 static esp_err_t http_status_handler(httpd_req_t *req)
267 {
268     httpd_resp_set_type(req, "application/json");
269     httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "");
270     char resp[160];
271     snprintf(resp, sizeof(resp),
272         "{\"device\":\"ESP32-S3\",\"ip\":\"%s\",\"mode\":\"%s\",\"led\":%d,\"port\":%d}",
273         s_device_ip, s_wifi_mode, s_led_state, TCP_SERVER_PORT);
274     httpd_resp_send(req, resp, HTTPD_RESP_USE_STRLEN);
275     return ESP_OK;
276 }
277
278 static const char INDEX_HTML[] =
279 "<!DOCTYPE html><html><head><meta name='viewport'
280 content='width=device-width,initial-scale=1'>"
281 "<title>ESP32-S3 Wireless</title><style>"
282 "body{font-family:system-ui,-apple-system,sans-serif;background:#0d1117;color:#c9d1d9;text-align:center;padding:20px;
283 margin:0}"
284 ".card{background:#161b22;border:1px solid
285 #30363d;border-radius:16px;max-width:380px;margin:auto;padding:24px;box-shadow:0 8px 24px rgba(0,0,0,0.5)}"
286 "h2{color:#58a6ff;margin:0 0 12px 0;font-size:22px}"
287 ".badge{display:inline-block;padding:6px 14px;border-radius:20px;font-size:14px;font-weight:700;margin-bottom:20px}"
288 ".on{background:rgba(46,160,67,0.25);color:#3fb950;border:1px solid #2ea043}"
289 ".off{background:rgba(248,81,73,0.25);color:#f85149;border:1px solid #da3633}"
290 ".btn{display:block;width:100%;padding:16px;margin:12px
291 0;font-size:18px;font-weight:700;border:none;border-radius:12px;cursor:pointer;transition:all 0.15s}"
```



```
346 .user_ctx = NULL
347 };
348 httpd_uri_t android_uri = {
349 .uri = "/generate_204",
350 .method = HTTP_GET,
351 .handler = http_root_handler,
352 .user_ctx = NULL
353 };
354 httpd_uri_t action_post = {
355 .uri = "/api/action",
356 .method = HTTP_POST,
357 .handler = http_action_handler,
358 .user_ctx = NULL
359 };
360 httpd_uri_t action_get = {
361 .uri = "/api/action",
362 .method = HTTP_GET,
363 .handler = http_action_handler,
364 .user_ctx = NULL
365 };
366 httpd_uri_t status_get = {
367 .uri = "/api/status",
368 .method = HTTP_GET,
369 .handler = http_status_handler,
370 .user_ctx = NULL
371 };
372 httpd_register_uri_handler(s_http_server, &root_uri);
373 httpd_register_uri_handler(s_http_server, &index_uri);
374 httpd_register_uri_handler(s_http_server, &apple_uri);
375 httpd_register_uri_handler(s_http_server, &android_uri);
376 httpd_register_uri_handler(s_http_server, &action_post);
377 httpd_register_uri_handler(s_http_server, &action_get);
378 httpd_register_uri_handler(s_http_server, &status_get);
379 ESP_LOGI(TAG, "Embedded HTTP Server listening on port 80 (Web UI & Captive Portal)");
380 }
381 }
382
383 // =====
384 // TCP Socket Server (Port 8888) - Transparent Wireless Serial Link
385 // =====
386 static void tcp_server_task(void *pvParameters)
387 {
388     char rx_buffer[128];
389     int addr_family = AF_INET;
390     int ip_protocol = IPPROTO_IP;
391
392     struct sockaddr_in dest_addr;
393     dest_addr.sin_addr.s_addr = htonl(INADDR_ANY);
394     dest_addr.sin_family = AF_INET;
395     dest_addr.sin_port = htons(TCP_SERVER_PORT);
396
397     int listen_sock = socket(addr_family, SOCK_STREAM, ip_protocol);
398     if (listen_sock < 0) {
399         ESP_LOGE(TAG, "Unable to create TCP socket: errno %d", errno);
400         vTaskDelete(NULL);
401         return;
402     }
403
404     int opt = 1;
```

```
405 setsockopt(listen_sock, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
406
407 int err = bind(listen_sock, (struct sockaddr *)&dest_addr, sizeof(dest_addr));
408 if (err != 0) {
409     ESP_LOGE(TAG, "TCP socket bind failed: errno %d", errno);
410     close(listen_sock);
411     vTaskDelete(NULL);
412     return;
413 }
414
415 err = listen(listen_sock, 2);
416 if (err != 0) {
417     ESP_LOGE(TAG, "TCP socket listen error: errno %d", errno);
418     close(listen_sock);
419     vTaskDelete(NULL);
420     return;
421 }
422
423 ESP_LOGI(TAG, "TCP Wireless Serial Server started on port %d", TCP_SERVER_PORT);
424
425 while (1) {
426     struct sockaddr_storage source_addr;
427     socklen_t addr_len = sizeof(source_addr);
428     int sock = accept(listen_sock, (struct sockaddr *)&source_addr, &addr_len);
429     if (sock < 0) {
430         ESP_LOGW(TAG, "Unable to accept connection: errno %d", errno);
431         vTaskDelay(pdMS_TO_TICKS(100));
432         continue;
433     }
434
435     s_active_client_sock = sock;
436     ESP_LOGI(TAG, "InnoIDE Client connected via TCP Socket!");
437
438     const char *welcome = "READY: InnoIDE Wireless Node Connected\r\n";
439     send(sock, welcome, strlen(welcome), 0);
440
441     char line_accum[128];
442     int accum_idx = 0;
443
444     while (1) {
445         int len = recv(sock, rx_buffer, sizeof(rx_buffer) - 1, 0);
446         if (len < 0) {
447             ESP_LOGW(TAG, "TCP recv error: errno %d", errno);
448             break;
449         } else if (len == 0) {
450             ESP_LOGI(TAG, "TCP connection closed by client");
451             break;
452         }
453
454         for (int i = 0; i < len; i++) {
455             char c = rx_buffer[i];
456             if (c == '\n' || c == '\r') {
457                 if (accum_idx > 0) {
458                     line_accum[accum_idx] = '\0';
459                     process_incoming_command(line_accum);
460                     accum_idx = 0;
461                 }
462             } else if (accum_idx < sizeof(line_accum) - 1) {
463                 line_accum[accum_idx++] = c;
```

```
464 }
465 }
466 }
467
468 if (s_active_client_sock == sock) {
469     s_active_client_sock = -1;
470 }
471 shutdown(sock, 0);
472 close(sock);
473 }
474
475 close(listen_sock);
476 vTaskDelete(NULL);
477 }
478
479 // =====
480 // UDP Discovery Beacon (Port 5056)
481 // Broadcasts an announcement so InnoIDE discovers this device automatically
482 // =====
483 static void udp_beacon_task(void *pvParameters)
484 {
485     struct sockaddr_in dest_addr;
486     dest_addr.sin_addr.s_addr = htonl(INADDR_BROADCAST);
487     dest_addr.sin_family = AF_INET;
488     dest_addr.sin_port = htons(UDP_BEACON_PORT);
489
490     int sock = socket(AF_INET, SOCK_DGRAM, IPPROTO_IP);
491     if (sock < 0) {
492         ESP_LOGE(TAG, "Unable to create UDP broadcast socket: errno %d", errno);
493         vTaskDelete(NULL);
494         return;
495     }
496
497     int broadcast_perm = 1;
498     setsockopt(sock, SOL_SOCKET, SO_BROADCAST, &broadcast_perm, sizeof(broadcast_perm));
499
500     while (1) {
501         if (s_wifi_connected) {
502             char beacon_msg[192];
503             snprintf(beacon_msg, sizeof(beacon_msg),
504                 "{ \"device\": \"ESP32-S3\", \"name\": \"ESP32-S3 Wireless Node\", \"ip\": \"%s\", \"port\": %d, \"mode\": \"%s\", \"led\": %d }",
505                 s_device_ip, TCP_SERVER_PORT, s_wifi_mode, s_led_state);
506
507             sendto(sock, beacon_msg, strlen(beacon_msg), 0, (struct sockaddr *)&dest_addr, sizeof(dest_addr));
508         }
509         vTaskDelay(pdMS_TO_TICKS(3000));
510     }
511
512     close(sock);
513     vTaskDelete(NULL);
514 }
515
516 // =====
517 // Captive Portal DNS Server (UDP Port 53)
518 // Resolves ALL domain names to 192.168.4.1 for instant mobile detection
519 // =====
520 static void dns_server_task(void *pvParameters)
521 {
522     uint8_t rx_buffer[128];
```

```
523 struct sockaddr_in dest_addr;
524 dest_addr.sin_addr.s_addr = htonl(INADDR_ANY);
525 dest_addr.sin_family = AF_INET;
526 dest_addr.sin_port = htons(53);
527
528 int sock = socket(AF_INET, SOCK_DGRAM, IPPROTO_IP);
529 if (sock < 0) {
530     vTaskDelete(NULL);
531     return;
532 }
533
534 if (bind(sock, (struct sockaddr *)&dest_addr, sizeof(dest_addr)) < 0) {
535     close(sock);
536     vTaskDelete(NULL);
537     return;
538 }
539
540 ESP_LOGI(TAG, "Captive Portal DNS Server started on UDP port 53");
541
542 while (1) {
543     struct sockaddr_storage source_addr;
544     socklen_t socklen = sizeof(source_addr);
545     int len = recvfrom(sock, rx_buffer, sizeof(rx_buffer), 0, (struct sockaddr *)&source_addr, &socklen);
546     if (len > 12) {
547         // Standard DNS Query -> Construct Answer pointing to 192.168.4.1
548         rx_buffer[2] |= 0x80; // QR = 1 (response)
549         rx_buffer[3] |= 0x80; // RA = 1 (recursion available)
550         rx_buffer[7] = 1; // ANCOUNT = 1
551
552         int resp_len = len;
553         rx_buffer[resp_len++] = 0xc0; // Name pointer
554         rx_buffer[resp_len++] = 0x0c;
555         rx_buffer[resp_len++] = 0x00; rx_buffer[resp_len++] = 0x01; // TYPE A
556         rx_buffer[resp_len++] = 0x00; rx_buffer[resp_len++] = 0x01; // CLASS IN
557         rx_buffer[resp_len++] = 0x00; rx_buffer[resp_len++] = 0x00; // TTL
558         rx_buffer[resp_len++] = 0x00; rx_buffer[resp_len++] = 0x3c;
559         rx_buffer[resp_len++] = 0x00; rx_buffer[resp_len++] = 0x04; // RDLLENGTH = 4
560         rx_buffer[resp_len++] = 192;
561         rx_buffer[resp_len++] = 168;
562         rx_buffer[resp_len++] = 4;
563         rx_buffer[resp_len++] = 1;
564
565         sendto(sock, rx_buffer, resp_len, 0, (struct sockaddr *)&source_addr, socklen);
566     }
567 }
568 close(sock);
569 vTaskDelete(NULL);
570 }
571
572 // =====
573 // Wi-Fi Connection Supervisor
574 // =====
575 static int s_sta_retry_count = 0;
576 #define MAX_RETRY_ATTEMPTS 4
577
578 static void wifi_event_handler(void *arg, esp_event_base_t event_base,
579 int32_t event_id, void *event_data)
580 {
581     if (event_base == WIFI_EVENT && event_id == WIFI_EVENT_STA_START) {
```

```
582     esp_wifi_connect();
583     ESP_LOGI(TAG, "Attempting connection to router SSID '%s'...", DEFAULT_WIFI_SSID);
584 } else if (event_base == WIFI_EVENT && event_id == WIFI_EVENT_STA_DISCONNECTED) {
585     wifi_event_sta_disconnected_t *disconn = (wifi_event_sta_disconnected_t *)event_data;
586     s_wifi_connected = 0;
587     int reason = disconn ? disconn->reason : 0;
588     if (s_sta_retry_count < MAX_RETRY_ATTEMPTS) {
589         s_sta_retry_count++;
590         ESP_LOGW(TAG, "STA reconnect attempt %d/%d (reason %d)...", s_sta_retry_count, MAX_RETRY_ATTEMPTS, reason);
591         esp_wifi_connect();
592     } else {
593         ESP_LOGI(TAG, "STA retries halted. SoftAP hotspot is 100%% active at 192.168.4.1");
594     }
595 } else if (event_base == IP_EVENT && event_id == IP_EVENT_STA_GOT_IP) {
596     ip_event_got_ip_t *event = (ip_event_got_ip_t *)event_data;
597     snprintf(s_device_ip, sizeof(s_device_ip), IPSTR, IP2STR(&event->ip_info.ip));
598     s_wifi_connected = 1;
599     s_sta_retry_count = 0;
600     strncpy(s_wifi_mode, "STA+AP_ACTIVE", sizeof(s_wifi_mode));
601     ESP_LOGI(TAG, "=====");
602     ESP_LOGI(TAG, " Wi-Fi Station Connected! IP: %s", s_device_ip);
603     ESP_LOGI(TAG, " Direct Hotspot IP: 192.168.4.1");
604     ESP_LOGI(TAG, " TCP Port: %d | HTTP Web UI Port: 80", TCP_SERVER_PORT);
605     ESP_LOGI(TAG, "=====");
606
607     char notify[96];
608     snprintf(notify, sizeof(notify), "WIFI:CONNECTED IP=%s PORT=%d\r\n", s_device_ip, TCP_SERVER_PORT);
609     send_serial_response(notify);
610 }
611 }
612
613 static void wifi_init_system(void)
614 {
615     esp_err_t ret = nvs_flash_init();
616     if (ret == ESP_ERR_NVS_NO_FREE_PAGES || ret == ESP_ERR_NVS_NEW_VERSION_FOUND) {
617         ESP_ERROR_CHECK(nvs_flash_erase());
618         ret = nvs_flash_init();
619     }
620     ESP_ERROR_CHECK(ret);
621
622     ESP_ERROR_CHECK(esp_netif_init());
623     ESP_ERROR_CHECK(esp_event_loop_create_default());
624
625     esp_netif_create_default_wifi_sta();
626     esp_netif_create_default_wifi_ap();
627
628     wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
629     ESP_ERROR_CHECK(esp_wifi_init(&cfg));
630
631     esp_event_handler_instance_t instance_any_id;
632     esp_event_handler_instance_t instance_got_ip;
633     ESP_ERROR_CHECK(esp_event_handler_instance_register(WIFI_EVENT,
634     ESP_EVENT_ANY_ID,
635     &wifi_event_handler,
636     NULL,
637     &instance_any_id));
638     ESP_ERROR_CHECK(esp_event_handler_instance_register(IP_EVENT,
639     IP_EVENT_STA_GOT_IP,
640     &wifi_event_handler,
```

```
641     NULL,
642     &instance_got_ip));
643
644     wifi_config_t sta_config = {
645         .sta = {
646             .ssid = DEFAULT_WIFI_SSID,
647             .password = DEFAULT_WIFI_PASS,
648             .threshold.authmode = WIFI_AUTH_OPEN,
649             .sae_pwe_h2e = WPA3_SAE_PWE_BOTH,
650         },
651     };
652
653     wifi_config_t ap_config = {
654         .ap = {
655             .ssid = FALLBACK_AP_SSID,
656             .ssid_len = strlen(FALLBACK_AP_SSID),
657             .channel = 1,
658             .password = FALLBACK_AP_PASS,
659             .max_connection = 4,
660             .authmode = WIFI_AUTH_WPA2_PSK
661         }
662     };
663
664     ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_APSTA));
665     ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_STA, &sta_config));
666     ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_AP, &ap_config));
667     ESP_ERROR_CHECK(esp_wifi_start());
668
669     strncpy(s_device_ip, "192.168.4.1", sizeof(s_device_ip));
670     strncpy(s_wifi_mode, "AP_HOTSPOT", sizeof(s_wifi_mode));
671
672     // Immediately launch HTTP server and DNS Captive Portal server
673     start_webserver();
674     xTaskCreate(dns_server_task, "dns_server", 2560, NULL, 5, NULL);
675
676     // Spin up background network tasks
677     xTaskCreate(tcp_server_task, "tcp_server", 4096, NULL, 5, NULL);
678     xTaskCreate(udp_beacon_task, "udp_beacon", 2560, NULL, 4, NULL);
679 }
680
681 // =====
682 // Application Entry Point
683 // Dual-Stack: USB CDC + Hardware UART0 + Wi-Fi operate concurrently!
684 // =====
685 void app_main(void)
686 {
687     ESP_LOGI(TAG, "=====");
688     ESP_LOGI(TAG, " InnoView Reactive Node Starting up...");
689     ESP_LOGI(TAG, " Channels: USB-CDC + UART0 + Wi-Fi TCP :8888 + HTTP :80");
690     ESP_LOGI(TAG, "=====");
691
692     // 1. Initialize Standard GPIO 2 Output
693     gpio_config_t io_conf = {
694         .pin_bit_mask = (1ULL << LED_GPIO_STANDARD),
695         .mode = GPIO_MODE_OUTPUT,
696         .pull_up_en = GPIO_PULLUP_DISABLE,
697         .pull_down_en = GPIO_PULLDOWN_DISABLE,
698         .intr_type = GPIO_INTR_DISABLE,
699     };

```

```
700 gpio_config(&io_conf);
701 gpio_set_level(LED_GPIO_STANDARD, 0);
702
703 // 2. Initialize WS2812 RGB LED Strip on GPIO 48
704 led_strip_config_t strip_config = {
705     .strip_gpio_num = LED_GPIO_WS2812,
706     .max_leds = LED_COUNT,
707     .led_model = LED_MODEL_WS2812,
708     .color_component_format = LED_STRIP_COLOR_COMPONENT_FMT_GRB,
709 };
710 led_strip_rmt_config_t rmt_config = {
711     .resolution_hz = 10 * 1000 * 1000,
712     .flags.with_dma = false,
713 };
714 esp_err_t err = led_strip_new_rmt_device(&strip_config, &rmt_config, &s_led_strip);
715 if (err != ESP_OK) {
716     ESP_LOGW(TAG, "WS2812 init warning: %s. Standard GPIO will still function.", esp_err_to_name(err));
717     s_led_strip = NULL;
718 }
719
720 // Explicitly ensure LED starts completely OFF on boot
721 set_hardware_led_state(0);
722
723 // 3. Initialize USB-Serial-JTAG driver for direct CDC serial commands (100% UNTOUCHED)
724 usb_serial_jtag_driver_config_t usb_config = {
725     .rx_buffer_size = 512,
726     .tx_buffer_size = 512,
727 };
728 usb_serial_jtag_driver_install(&usb_config);
729
730 // 4. Initialize UART0 driver for hardware UART serial commands (100% UNTOUCHED)
731 uart_config_t uart_config = {
732     .baud_rate = 115200,
733     .data_bits = UART_DATA_8_BITS,
734     .parity = UART_PARITY_DISABLE,
735     .stop_bits = UART_STOP_BITS_1,
736     .flow_ctrl = UART_HW_FLOWCTRL_DISABLE,
737     .source_clk = UART_SCLK_DEFAULT,
738 };
739 uart_driver_install(UART_NUM_0, 512, 512, 0, NULL, 0);
740 uart_param_config(UART_NUM_0, &uart_config);
741
742 // 5. Start background LED controller task (100% UNTOUCHED)
743 xTaskCreate(led_controller_task, "led_controller", 2048, NULL, 5, NULL);
744
745 // 6. Initialize Wi-Fi and Networking in background
746 wifi_init_system();
747
748 send_serial_response("READY: InnoIDE Reactive Firmware Ready (LED=OFF)\r\n");
749
750 char usb_buf[64];
751 int usb_idx = 0;
752 char uart_buf[64];
753 int uart_idx = 0;
754
755 // Continuous poll loop for wired USB CDC & UART0 (100% UNTOUCHED)
756 while (1)
757 {
758     // Check USB-Serial/JTAG stream
```

